Assignment: Multimodal Chat Assistant

Timeline: 1 Week

Problem Statement

Develop a **chat assistant for visual understanding** that can process video input, recognize events, summarize content based on guidelines, and engage in multi-turn conversations. The solution should be performant, handling video streams efficiently with low latency.

Objectives

By the end of this assignment, you will have designed and implemented a core system for a visual understanding chat assistant, demonstrating proficiency in video processing, event recognition, summarization, and conversational AI.

Features to Implement

Core Features

1. **Video Event Recognition & Summarization w.r.t. Guideline Adherence:**
    - The system must accept a video stream as input.
    - It should be capable of **identifying specific events** within the video.
    - Based on pre-defined guidelines (e.g., traffic rules for a traffic scene video), the system should **detect and report adherence or violations**.
    - Finally, it should **summarize the video content**, highlighting key events and any detected guideline adherence/violations.
    - **Example Scenario:** Given a traffic scene video, the assistant should identify vehicle movements, pedestrian crossings, traffic light changes, and then summarize traffic violations (e.g., "Vehicle X ran a red light at timestamp Y," "Pedestrian crossed against the signal at timestamp Z").
2. **Multi-Turn Conversations:**
    - The chat assistant must support **natural, multi-turn interactions** with the user. This means it should retain context from previous turns to understand follow-up questions and provide coherent responses.
    - Users should be able to ask clarifying questions or further inquire about events/summaries generated from the video.
3. **Video Input Processing:**
    - The system must be able to process an input video stream with a **maximum duration of 10 minutes** at a **frame rate of 60 frames per second (fps)**.

    - Consider efficient ways to handle this high-throughput video data, perhaps by processing frames in batches or intelligently sampling.

Bonus Features (Optional, for scoring additional points)

1. **VLM/Cognition Model with Unlimited Input Tokens:**
    - Integrate or utilize a Vision-Language Model (VLM) or cognition model that effectively handles **very large input contexts**, ideally supporting **greater than 512k tokens**. This is crucial for processing long video summaries or detailed multi-turn conversations without losing context.
    - (Consider models that employ techniques like hierarchical attention, sparse attention, or external memory.)
2. **Low Latency:**
    - Achieve a system **latency of less than 2 seconds** for processing queries and generating responses after the video input is provided. This implies efficient real-time or near-real-time processing and inference.

Choice of Tools

- **Backend Tech Stack:** You have the freedom to choose any backend technology stack (e.g., Python with FastAPI/Flask, Node.js with Express, Go, etc.). Justify your choice based on its suitability for AI/ML workloads, real-time processing, and scalability.
- **VLM API:** Prioritize the use of **open-source VLM APIs or models**. If open-source options are limited for the specific features required, you may consider readily available API services, but clearly state your reasoning.

Deliverables

By the end of the week, submit the following:

1. **Source Code:** A well-organized and commented codebase for your chat assistant.
2. **README.md File:** A comprehensive README file including:
    - **Project Overview:** A brief description of your solution.
    - **Architecture Diagram:** A high-level diagram illustrating the components of your system and their interactions.
    - **Tech Stack Justification:** Explanation for your chosen backend technologies and VLM.
    - **Setup and Installation Instructions:** Clear steps to get the project running.

    - **Usage Instructions:** How to interact with the chat assistant, including examples of video input and conversational queries.
    - **Demo Video (Optional but highly recommended):** A short video demonstrating the features of your chat assistant.
3. **Challenges and Learnings:** A section documenting any significant challenges encountered during development and the solutions or workarounds implemented. Also, reflect on key learnings from this assignment.

Evaluation Criteria

Your assignment will be evaluated based on:

- **Functionality:** How well the core features (video event recognition, summarization, multi-turn conversations) are implemented and performed. Few test samples are at https://drive.google.com/drive/folders/1RYil2fWSlkKsmf65CwSav3H4h5DDHikl
- **Code Quality:** Readability, modularity, comments, and adherence to good coding practices.
- **System Design:** The clarity and robustness of your architectural design.
- **Performance:** Efficiency in processing video, especially concerning the 60 fps requirement, and latency for bonus points.
- **Innovation/Creativity:** Any unique approaches or extra functionalities (beyond bonus features) that enhance the assistant's capabilities.
- **Documentation:** Completeness and clarity of your `README.md` file.